

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №1
по дисциплине «Интеллектуальные методы анализа данных»
Тема: Первичный анализ и предобработка природного временного
ряда.

Студент гр. 3384

Козьмин Н.В.

Преподаватель

Мандрикова Б.С.

Санкт-Петербург

2026

Цель работы.

Повышение качества природных данных на примере наблюдений нейтронных мониторов

Задание.

Вариант 8.

Наименование станции: Тикси (TXBY). Период: 20-29 сентября 2015

1. Скачать данные из источника [<https://www.nmdb.eu/nest/>] за периоды и станции, указанные в варианте. Использовать данные с одноминутным временным разрешением.
2. Выполнить обнаружение пропусков и выбросов (3σ) в анализируемом временном ряде, посчитать их количество и долю от всех обрабатываемых данных.
3. С помощью функции `median` в MatLab (или аналогичной из другого инструмента) вычислить медианное значение по данным без пропусков и выбросов.
4. Заменить пропуски и выбросы медианными значениями.
5. Изобразить графики исходного и полученного временных рядов.
6. Сделать соответствующие выводы, оформить отчет по работе.

Выполнение работы.

Для выполнения лабораторной работы был подготовлен программный алгоритм обработки одномоментного временного ряда нейтронного монитора. Обработка выполнялась последовательно: сначала данные были загружены из внешнего источника, затем приведены к единому временному формату, проверены на пропуски и выбросы, очищены медианной заменой и представлены графически. Полный исходный код приведён в приложении А.

Загрузка данных из NMDB NEST

Сначала данные были загружены непосредственно из базы NMDB NEST с помощью библиотеки requests. В URL-запросе (взятом с сайта) была указана станция TXBY, соответствующая станции Тикси, а также период с 20 сентября 2015 года 00:00 UTC по 29 сентября 2015 года 23:59 UTC. Параметр `tresolution=0` задаёт одномоментное временное разрешение. Параметр `dtype=corr_for_efficiency` определяет используемый показатель `RCORR_E` — ряд, скорректированный с учётом эффективности детектора. Параметр `output=ascii` запрашивает текстовый табличный ответ, удобный для последующей обработки.

После отправки запроса был проверен статус ответа сервера. Из полученного ASCII-ответа были выбраны строки, содержащие временную метку и числовое значение, а служебные строки и заголовки NEST в анализ не включались. Затем текст был преобразован в таблицу pandas с двумя столбцами: `datetime` и `corr_for_efficiency`.

Подготовка данных и используемые переменные

Столбец `datetime` был преобразован функцией `pandas.to_datetime` с указанием временной зоны UTC. Значения показателя были преобразованы в числовой тип функцией `pandas.to_numeric`. Некорректные записи были удалены, а оставшиеся наблюдения отсортированы по времени. В результате было получено 14 032 корректных наблюдения.

Переменная `url` хранит адрес запроса к NEST, `response` содержит ответ сервера, а `data_lines` — отобранные строки измерений. Таблица `df` содержит

исходные наблюдения. Переменная `full_index` содержит полную минутную временную сетку, `series` — ряд после сопоставления измерений с этой сеткой, а `available` — доступные значения без временных пропусков. Логические маски `missing_mask` и `outlier_mask` отмечают пропуски и выбросы. Переменная `median_value` хранит рассчитанную медиану, а `cleaned` содержит итоговый очищенный ряд.

Обнаружение пропусков

Для обнаружения пропусков была восстановлена полная минутная сетка с помощью функции `pandas.date_range`. В неё вошли все минуты от 20.09.2015 00:00 до 29.09.2015 23:59 UTC включительно. После этого исходные наблюдения были переиндексированы относительно полной сетки. Если для некоторой минуты запись отсутствовала, после переиндексации появлялось значение NaN, которое обнаруживалось выражением `series.isna()`. Такой подход позволяет найти не только пустые значения, но и полностью отсутствующие строки между двумя соседними измерениями.

За десять суток ожидалось 14400 временных позиций, а доступно было 14032 наблюдений. Поэтому обнаружено 368 пропусков. Их доля вычислена программно относительно полной минутной сетки: $368 / 14400 \cdot 100\% = 2,556\%$.

Обнаружение выбросов

Для обнаружения выбросов использовался критерий трёх стандартных отклонений. Сначала по доступным значениям были рассчитаны среднее значение μ и стандартное отклонение σ . Наблюдение считалось выбросом, если выполнялось условие:

$$|x - \mu| > 3\sigma$$

Для рассматриваемого ряда среднее значение составило $\mu = 85,835529$, стандартное отклонение — $\sigma = 282,212271$. Нижняя граница критерия равна - 760,801283, верхняя граница — 932,472342. Проверка каждого доступного значения выявила 14 выбросов. Все найденные выбросы превышали верхнюю границу и соответствовали аномально большим пикам исходного ряда.

Доля выбросов также вычислена программно: $14 / 14400 \cdot 100\% = 0,097\%$.

Вычисление медианы и замена значений

После обнаружения пропусков и выбросов из ряда были исключены значения, отмеченные соответствующими логическими масками. По оставшимся корректным наблюдениям была вычислена медиана функцией `median` из библиотеки `pandas`. Медиана использована потому, что она устойчивее среднего значения к экстремальным наблюдениям и лучше характеризует типичный уровень ряда при наличии крупных пиков.

Медианное значение составило 86,133. Затем все позиции, для которых отсутствовало измерение или была установлена принадлежность к выбросам, были заменены этим значением. Заменено 368 пропусков и 14 выбросов, всего — 382 значений.

Общая доля замен вычислена программно: $382 / 14400 \cdot 100\% = 2,653\%$. После замены выполнена контрольная проверка `cleaned.isna().sum()`. Результат проверки равен нулю, следовательно, итоговый ряд не содержит пропущенных значений.

Построение и анализ графиков

Для визуального сравнения исходного и обработанного рядов использовалась функция `matplotlib.pyplot.subplots`, создающая два графика с общей временной осью. На верхнем графике показан исходный ряд `series`, содержащий пропуски и выбросы. На нижнем графике показан ряд `cleaned` после медианной замены. Были добавлены подписи осей, заголовки и сетка. Результат представлен на рисунке 1.

Результат сравнения исходного и очищенного временных рядов представлен на рисунке 1.

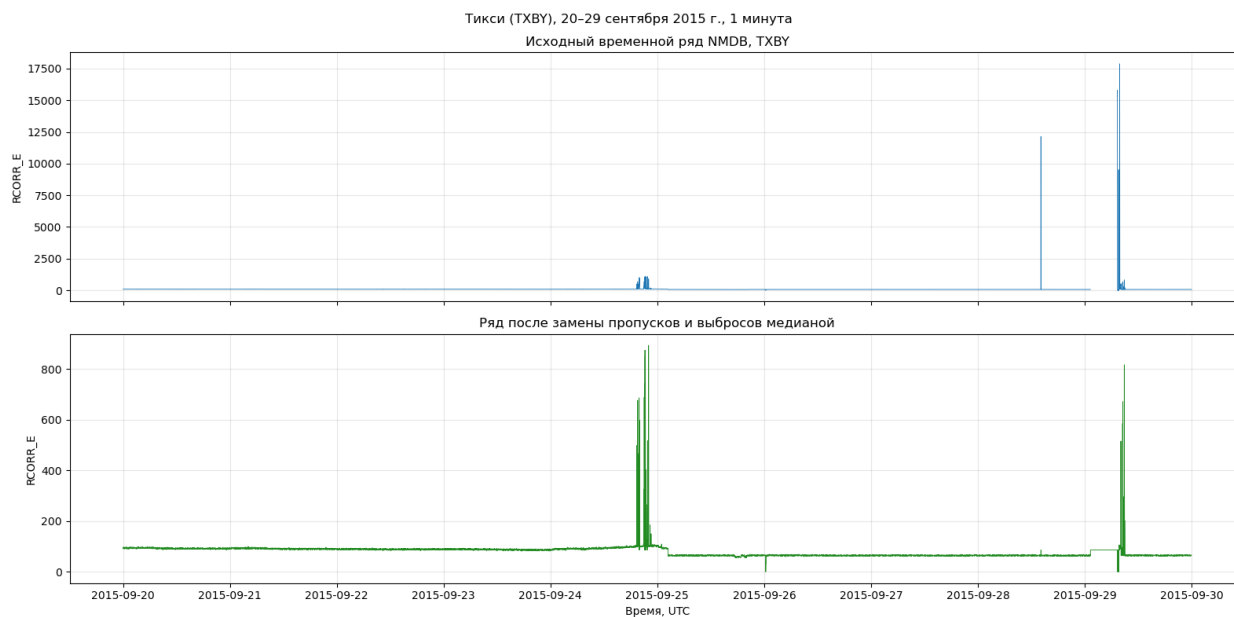


Рисунок 1 – Исходный и очищенный временные ряды станции TXBY

На исходном графике видны разрывы, соответствующие отсутствующим минутам, и отдельные резкие пики, признанные выбросами по критерию 3σ . После обработки ряд становится непрерывным: пропуски заполнены медианой, а экстремальные значения заменены тем же устойчивым статистическим уровнем.

Разработанный программный код приведён в приложении А.

Выводы.

В ходе лабораторной работы было исследовано качество природного временного ряда наблюдений нейтронного монитора станции Тикси за период с 20 по 29 сентября 2015 года. Цель работы достигнута: исходные данные были загружены из NMDB, проверены, очищены от пропусков и выбросов и представлены в виде графиков.

В полном десятисуточном интервале содержится 14400 минутных позиций. Из NEST было получено 14032 наблюдений. Обнаружено 368 пропусков и 14 выбросов. Медиана по корректным данным составила 86,133. Всего медианными значениями заменено 382 позиций, что соответствует 2,653% полного временного ряда.

Количество, доли и медианное значение формируются программно при запуске ноутбука, поэтому результаты можно воспроизвести для того же набора данных. После обработки пропуски отсутствуют, и полученный непрерывный ряд пригоден для последующего статистического анализа.

Важными являются следующие гипотезы:

1. Так как выбросы остаются относительно большими, имеет смысл уменьшить коэффициент при сигме. Например, до 0.5.
2. Если визуально аппроксимировать график, то можно утверждать, что для заполнения пропусков 30 сентября лучше бы подошла медиана, посчитанная для последовательности от начала 26 сентября.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Название файла: main.ipynb

```
from io import StringIO
from pathlib import Path
import requests
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

ROOT = Path.cwd()
url =
r'''https://www.nmdb.eu/nest/draw_graph.php?formchk=1&stations[]=TXB
Y&tabchoice=revori&dtype=corr_for_efficiency&tresolution=0&yunits=0&
date_choice=bydate&start_day=20&start_month=9&start_year=2015&start_
hour=0&start_min=0&end_day=29&end_month=9&end_year=2015&end_hour=23&
end_min=59&output=ascii'''
response = requests.get(url, timeout=60)
response.raise_for_status()

data_lines = [line.strip() for line in response.text.splitlines() if
line[:4].isdigit() and ';' in line]
df = pd.read_csv(StringIO('\n'.join(data_lines)), sep=';',
names=['datetime', 'corr_for_efficiency'])
df['datetime'] = pd.to_datetime(df['datetime'], utc=True)
df['corr_for_efficiency'] = pd.to_numeric(df['corr_for_efficiency'],
errors='coerce')
df = df.dropna().sort_values('datetime').reset_index(drop=True)
print(f'Загружено строк из NMDB: {len(df)}')

df.head(10)

df['datetime'] = pd.to_datetime(df['datetime'], utc=True)
df['corr_for_efficiency'] = pd.to_numeric(df['corr_for_efficiency'],
errors='coerce')
df =
df.dropna().sort_values('datetime').drop_duplicates('datetime').rese
t_index(drop=True)

start = pd.Timestamp('2015-09-20 00:00:00', tz='UTC')
end = pd.Timestamp('2015-09-29 23:59:00', tz='UTC')
full_index = pd.date_range(start, end, freq='min')
series =
df.set_index('datetime')['corr_for_efficiency'].reindex(full_index)
series.index.name = 'datetime'
print(f'Ожидаемых минут: {len(full_index)}')
print(f'Полученных наблюдений: {series.notna().sum()}')
print(f'Пропусков: {series.isna().sum()}')

available = series.dropna()
mu = available.mean()
sigma = available.std(ddof=0) # смещенная оценка
```

```

lower_bound = mu - 3 * sigma
upper_bound = mu + 3 * sigma
outlier_mask = series.notna() & ((series < lower_bound) | (series >
upper_bound))
print(f"Среднее: {mu:.6f}")
print(f"Стандартное отклонение: {sigma:.6f}")
print(f"Границы 3σ: [{lower_bound:.6f}; {upper_bound:.6f}]")
print(f"Выбросов: {outlier_mask.sum()}")

```

```

missing_mask = series.isna()
median_value = available[~outlier_mask].median()
cleaned = series.mask(missing_mask | outlier_mask, median_value)
print(f"Медиана без пропусков и выбросов: {median_value:.3f}")
print(f"Заменено пропусков: {missing_mask.sum()}")
print(f"Заменено выбросов: {outlier_mask.sum()}")
print(f"Всего замен: {(missing_mask | outlier_mask).sum()}")
print(f"Пропусков после обработки: {cleaned.isna().sum()}")

```

```

fig, axes = plt.subplots(2, 1, figsize=(16, 8), sharex=True)
axes[0].plot(series.index, series, linewidth=0.65)
axes[0].set_title("Исходный временной ряд NMDB, TXBY")
axes[0].set_ylabel("RCORR_E")
axes[0].grid(True, alpha=0.3)
axes[1].plot(cleaned.index, cleaned, linewidth=0.65,
color="forestgreen")
axes[1].set_title("Ряд после замены пропусков и выбросов медианой")
axes[1].set_ylabel("RCORR_E")
axes[1].set_xlabel("Время, UTC")
axes[1].grid(True, alpha=0.3)
fig.suptitle("Тикси (TXBY), 20-29 сентября 2015 г., 1 минута")
fig.tight_layout()
plt.show()

```

```

total_count = len(full_index)
observed_count = int(series.notna().sum())
missing_count = int(missing_mask.sum())
outlier_count = int(outlier_mask.sum())
replaced_count = int((missing_mask | outlier_mask).sum())
summary = pd.DataFrame({
    'Показатель': ['Полный ряд', 'Полученные наблюдения',
'Пропуски', 'Выбросы', 'Всего замен', 'Медиана'],
    'Количество / значение': [total_count, observed_count,
missing_count, outlier_count, replaced_count, median_value],
    'Доля от полного ряда, %': [100.0, observed_count / total_count
* 100, missing_count / total_count * 100, outlier_count /
total_count * 100, replaced_count / total_count * 100, np.nan]
})
summary.style.format({'Количество / значение': '{:.3f}', 'Доля от
полного ряда, %': '{:.3f}'})
display(summary)

```